

Performance Testing

Date	15 July 2026
Team ID	
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the OptiCrop system behaves under expected and peak load conditions. The sections below document the testing approach, tools used, and results obtained for the Flask-based crop recommendation application.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman (API testing) + Browser-based manual testing + Flask Debug Server
Type of Testing	Functional Testing, Load Testing (manual), Response Time Testing, Accuracy Testing
Target Module / API	/predict endpoint (POST) — Crop recommendation prediction route
Test Environment	Local Development Server — Flask (http://127.0.0.1:5000)
Test Date	26 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Home page load test — access http://127.0.0.1:5000	1	< 2	Page loads successfully with navigation
2	Crop prediction with valid inputs (N=105, P=35, K=40, T=25, H=64, pH=7, R=160)	1	< 2	Returns "coffee" as recommended crop
3	Crop prediction with different valid inputs (Rice conditions)	1	< 2	Returns correct crop recommendation
4	Multiple consecutive prediction requests to test response consistency	5	< 5	Consistent results returned for same inputs

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	~0.8 seconds	Pass	Fast local prediction
2	Response Time (Max)	< 5 seconds	~1.2 seconds	Pass	Well within limit
3	Throughput (Req/sec)	> 5 req/sec	~8 req/sec	Pass	Sufficient for local use

4	Error Rate	< 1%	0%	Pass	No errors during testing
5	CPU Utilization	< 80%	~15%	Pass	Minimal resource usage
6	Memory Utilization	< 80%	~12%	Pass	Model loaded efficiently

Step 4: Observations & Analysis

Key Findings: The OptiCrop Flask application performs efficiently on local development server. Prediction response time averages 0.8 seconds, well within the 2-second target. The Logistic Regression model achieves 94% accuracy on test data. Bottlenecks Identified: The application is currently limited to local deployment. Performance may vary with concurrent users in production environment. Model loading from pickle file adds minor startup time. Optimization Steps Taken: Model is pre-loaded at application startup (not per request) to minimize response time. `warnings.filterwarnings()` suppresses unnecessary sklearn convergence warnings. Input validation ensures only valid numerical values reach the prediction model.